

Practical: MNIST Handwritten Digit Detection using TensorFlow and Keras

Aim

To implement MNIST Handwritten Character Detection using TensorFlow and Keras.

Objective

To study:

- 1. MNIST dataset**
 - 2. Convolutional Neural Network (CNN)**
 - 3. TensorFlow and Keras framework**
 - 4. Handwritten digit recognition**
-

Important Theory for Practical Writing

What is MNIST Dataset?

MNIST is a standard handwritten digit dataset used in machine learning and deep learning.

It contains:

- 60,000 training images**
- 10,000 testing images**

Each image:

- Contains handwritten digits from 0 to 9**
- Size = 28×28 pixels**
- Grayscale image**

MNIST is called the “Hello World” dataset of deep learning because it is simple and easy to use.

What is TensorFlow?

TensorFlow is an open-source deep learning framework developed by Google.

It is used for:

- Machine learning**
 - Neural networks**
 - Image processing**
 - AI applications**
-

What is Keras?

Keras is a high-level API of TensorFlow.

It is used to:

- Build neural networks easily
 - Train deep learning models
 - Simplify coding
-

What is CNN?

CNN stands for Convolutional Neural Network.

CNN is mainly used for:

- Image recognition
- Image classification
- Object detection

CNN automatically extracts important image features.

Layers Used in CNN

1. Convolution Layer

Extracts image features like:

- edges
 - curves
 - patterns
-

2. MaxPooling Layer

Reduces image size and computation.

3. Flatten Layer

Converts 2D image data into 1D vector.

4. Dense Layer

Performs classification.

Softmax Activation Function

Softmax converts outputs into probabilities.

Formula:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Output probabilities sum to 1.

Loss Function

The model uses Sparse Categorical Crossentropy loss.

Used for:

- Multi-class classification problems
-

Optimizer

Adam optimizer is used.

Purpose:

- Update weights
 - Reduce loss
 - Improve accuracy
-

Applications

1. Digit recognition
2. OCR systems

Code

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

# Normalize the data (scale pixel values to 0-1)
X_train, X_test = X_train / 255.0, X_test / 255.0
```

```
# Build a simple CNN model
```

```
model = keras.Sequential([  
    keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),  
    keras.layers.MaxPooling2D(2,2),  
  
    keras.layers.Conv2D(64, (3,3), activation='relu'),  
    keras.layers.MaxPooling2D(2,2),  
  
    keras.layers.Flatten(),  
  
    keras.layers.Dense(128, activation='relu'),  
    keras.layers.Dense(10, activation='softmax') # 10 classes (digits 0-9)  
])
```

```
# Compile the model
```

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
# Train the model
```

```
model.fit(  
    X_train,  
    y_train,  
    epochs=5,  
    validation_data=(X_test, y_test)  
)
```

```
# Evaluate the model
```

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
```

```
print("Test Accuracy:", test_accuracy)
```

```
# Predict a random sample
index = np.random.randint(0, len(X_test))

plt.imshow(X_test[index], cmap='gray')
plt.title(f"True Label: {y_test[index]}")
plt.show()

prediction = np.argmax(
    model.predict(X_test[index].reshape(1, 28, 28, 1))
)

print("Predicted Label:", prediction)
```